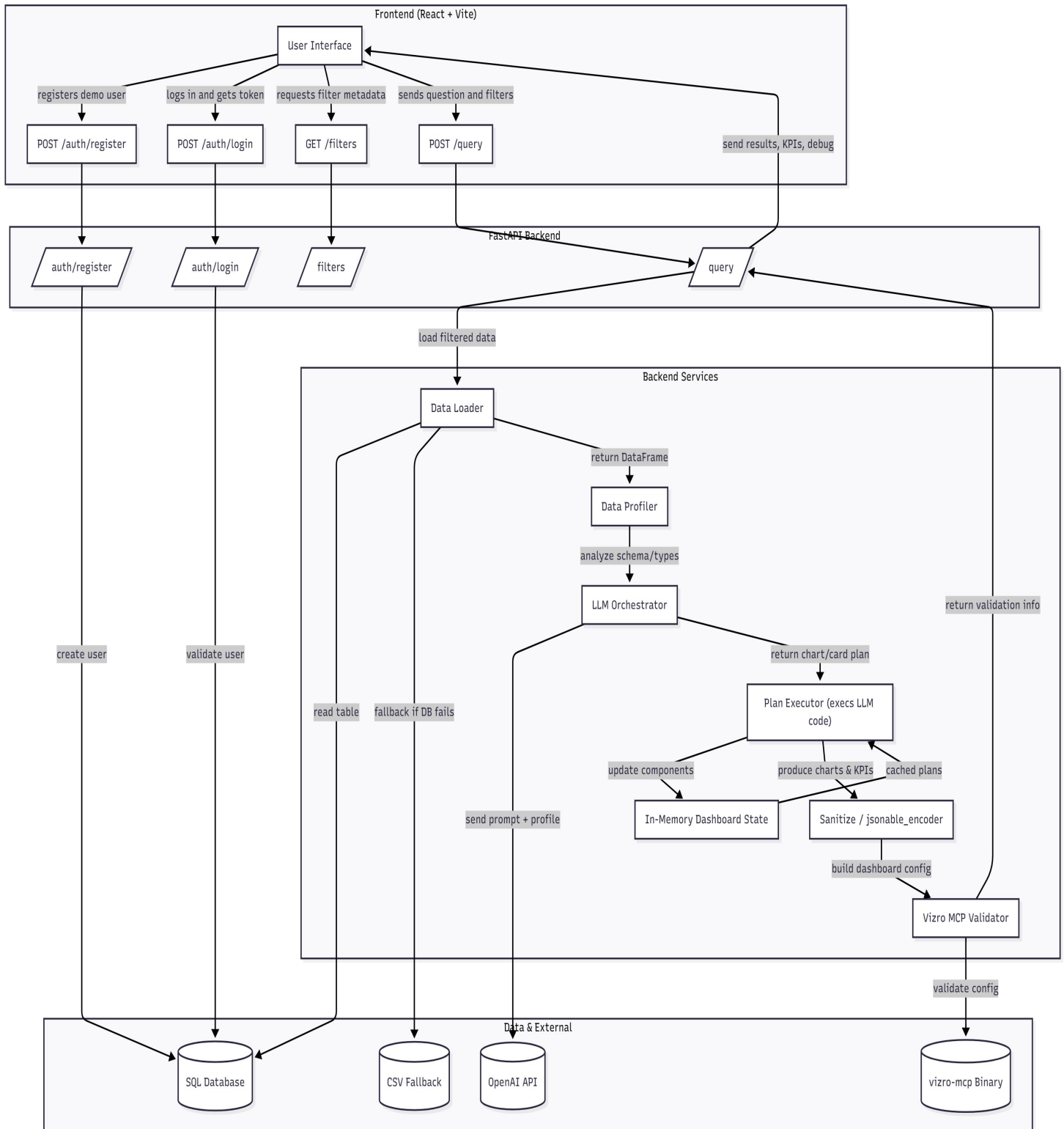


P13 - ContinuumAi: Updated System Architecture

This document highlights the updated system architecture for ContinuumAi. As stated in the README.md as well, sprint 1 consisted of us continuing our old non-vizro application, after the research phase we shifted to Vizro MCP, so technically there were 2 simultaneous systems running in parallel: the old one and the new one. The architecture below is for the Vizro MCP system, which is the one we will be carrying forward in sprint2 and so on.



Summary:

This application is a personal sales business analyst that converts natural-language questions into validated, interactive dashboards. It is built as a single-page chatbot styled React (Vite + TypeScript) frontend and a FastAPI backend that orchestrates data access, LLM-driven plan generation, safe execution of visualization code, and dashboard validation via Vizro MCP.

Quick Workflow

User asks a question in the frontend → backend loads & profiles filtered data → LLM produces chart/card plans → backend executes plans to produce Plotly KPIs and builds a Vizro dashboard → Vizro validates the config → frontend receives results + debug info and renders the dashboard.

Components & responsibilities

Frontend (React + Vite) - bootstraps a demo user, stores the JWT, fetches filter metadata, sends user questions + filters to the backend, and renders returned Plotly JSON charts and KPI cards. Also exposes debug/code returned by the backend for inspection.

API (FastAPI) - authentication endpoints (/auth/*), public filter endpoint (/filters), and protected query endpoint (/query). Orchestrates data loading, profiling, LLM calls, plan execution, dashboard assembly, validation, and response serialization.

Data layer - primary SQL database (via SQLAlchemy) for users and optional demo table, with a CSV fallback for resilience. DataLoader applies requested filters and returns a pandas DataFrame for downstream processing.

LLM orchestration - builds a system prompt from the DataFrame profile and calls OpenAI to produce a structured chart_plan and card_plan (code + metadata). The backend parses that JSON and treats it as the source of visualization logic.

Plan executor & visualization - executes plan code in a controlled environment (pd, plotly, df available) to produce Plotly figures and KPI outputs; those results are sanitized and assembled into a Vizro dashboard config.

Vizro MCP validator - validates the generated dashboard config (using profiler-derived data_infos) to catch schema mismatches and surface validation messages back to the frontend.

In-memory state - a lightweight process-local store (custom charts, card plans, caches, LLM history) used to persist and re-render components across queries during the backend process lifetime.